### 1．Шаблоны проектирования и архитектурные шаблоны — 设计模式与架构模式

Шаблон проектирования — это типовое решение повторяющейся задачи проектирования программного обеспечения. Он описывает взаимодействие классов и объектов, но не является готовым кодом. Выделяют порождающие, структурные и поведенческие шаблоны. Например, фабрика относится к порождающим шаблонам, адаптер — к структурным, а стратегия — к поведенческим.

设计模式是针对软件设计中反复出现的问题的典型解决方案。它描述类和对象之间的协作方式，但不是现成的代码。设计模式分为创建型、结构型和行为型。例如，工厂属于创建型模式，适配器属于结构型模式，策略属于行为型模式。

Архитектурные шаблоны определяют общую организацию приложения. Примеры — многослойная архитектура, MVC и микросервисная архитектура. Архитектурный шаблон определяет структуру приложения в целом, а шаблон проектирования обычно решает более локальную задачу взаимодействия классов и объектов.

架构模式规定应用的整体组织方式。例如，分层架构、MVC 和微服务架构。架构模式规定整个应用的结构，而设计模式通常解决更局部的类和对象协作问题。

В моей работе используется многослойная архитектура. MovieResource принимает HTTP-запросы, MovieService выполняет бизнес-логику, а Database управляет доступом к данным и транзакциями. Такое разделение упрощает сопровождение и тестирование приложения. Также используется внедрение зависимостей: MovieResource получает MovieService через аннотацию Inject.

我的实验采用分层架构。MovieResource 接收 HTTP 请求，MovieService 执行业务逻辑，Database 管理数据访问和事务。这样的划分简化了应用的维护与测试。系统还使用依赖注入：MovieResource 通过 Inject 注解获得 MovieService。

### 2．Платформа Jakarta EE. Виды компонентов — Jakarta EE 平台与组件类型

Jakarta EE — это платформа для разработки корпоративных Java-приложений, основанная на наборе спецификаций. Она является преемником Java EE и работает поверх Java SE. Платформа определяет стандартные API для веб-приложений, внедрения зависимостей, работы с данными, транзакций, безопасности и обмена сообщениями.

Jakarta EE 是一个基于一组规范、用于开发企业级 Java 应用的平台。它是 Java EE 的后继者，建立在 Java SE 的基础之上。该平台为 Web 应用、依赖注入、数据访问、事务、安全和消息传递定义标准 API。

К компонентам относятся сервлеты, REST-ресурсы, CDI-бины и Enterprise Beans. Для представления постоянных предметных данных используются сущности Jakarta Persistence. Контейнер управляет жизненным циклом соответствующих компонентов и предоставляет им необходимые службы.

组件包括 Servlet、REST 资源、CDI Bean 和 Enterprise Bean。Jakarta Persistence 实体用于表示需要持久化的领域数据。容器管理相应组件的生命周期，并为它们提供所需服务。

Jakarta EE не является сервером приложений. Она определяет спецификации, а сервер приложений предоставляет их реализации. В моей работе приложение запускается в Payara. Используются Jakarta REST, CDI, Jakarta Persistence и Bean Validation.

Jakarta EE 不是应用服务器。它定义规范，而应用服务器提供这些规范的实现。我的实验在 Payara 中运行，使用 Jakarta REST、CDI、Jakarta Persistence 和 Bean Validation。

### 3．Управляемые бины. CDI-бины — 受管理 Bean 与 CDI Bean

Управляемый бин — это компонент, жизненным циклом которого управляет контейнер. CDI означает Contexts and Dependency Injection, то есть контексты и внедрение зависимостей. CDI обеспечивает внедрение зависимостей и управление областями действия бинов.

受管理 Bean 是生命周期由容器管理的组件。CDI 是 Contexts and Dependency Injection 的缩写，即上下文与依赖注入。CDI 提供依赖注入和 Bean 作用域管理。

Аннотация Inject указывает место внедрения зависимости. Контейнер выбирает подходящий бин и предоставляет ссылку на него. RequestScoped означает область одного HTTP-запроса, SessionScoped — область пользовательской сессии, а ApplicationScoped — область приложения.

Inject 注解声明依赖注入的位置。容器选择合适的 Bean，并提供对它的引用。RequestScoped 表示一次 HTTP 请求的作用域，SessionScoped 表示用户会话作用域，ApplicationScoped 表示应用作用域。

В моей работе MovieResource имеет область RequestScoped, а MovieService и Database — ApplicationScoped. Метод с аннотацией PostConstruct выполняет инициализацию после внедрения зависимостей, а метод с PreDestroy освобождает ресурсы перед уничтожением компонента.

我的实验中，MovieResource 使用 RequestScoped，MovieService 和 Database 使用 ApplicationScoped。标有 PostConstruct 的方法在依赖注入后执行初始化，标有 PreDestroy 的方法在组件销毁前释放资源。

Я использую внедрение зависимостей вместо самостоятельного создания сервисов через new. Их жизненным циклом и зависимостями управляет CDI-контейнер. Обычный вызов new сам по себе не выполняет CDI-внедрение.

我使用依赖注入，而不是通过 new 自行创建服务对象。它们的生命周期和依赖由 CDI 容器管理。普通的 new 调用本身不会执行 CDI 依赖注入。

**4．ORM. Hibernate и EclipseLink — ORM、Hibernate 与 EclipseLink**

ORM — это объектно-реляционное отображение. Оно связывает объектную модель приложения с реляционной моделью базы данных. Класс сущности обычно соответствует таблице, объект — записи, а его поля — столбцам или связям.

ORM 指对象关系映射。它将应用的对象模型与数据库的关系模型联系起来。实体类通常对应表，对象对应记录，而字段对应列或关联关系。

Hibernate и EclipseLink — это ORM-провайдеры, реализующие Jakarta Persistence. Оба поддерживают отображение сущностей, связи, запросы, отслеживание изменений и блокировки. Они отличаются внутренней реализацией, настройками и дополнительными возможностями. Hibernate предоставляет собственный API Session, а EclipseLink — собственные расширения стандартного API.

Hibernate 和 EclipseLink 是实现 Jakarta Persistence 的 ORM Provider。两者都支持实体映射、关联关系、查询、变更跟踪和锁。它们在内部实现、配置和扩展功能方面存在差异。Hibernate 提供自己的 Session API，EclipseLink 则提供对标准 API 的自身扩展。

В моей работе используется EclipseLink, поскольку это прямо требуется заданием. Основные операции выполняются через стандартный API JPA. Например, связь между фильмом и координатами описывается в Java как ссылка на объект, а в базе данных хранится с помощью внешнего ключа.

我的实验使用 EclipseLink，因为任务明确要求使用它。主要操作通过标准 JPA API 完成。例如，电影与坐标之间的关系在 Java 中表现为对象引用，在数据库中则通过外键保存。

ORM упрощает доступ к данным, но не отменяет необходимость понимать SQL. Разработчик должен понимать связи, индексы и транзакции, чтобы контролировать корректность и производительность запросов.

ORM 简化数据访问，但并不意味着不再需要理解 SQL。开发者必须理解关联关系、索引和事务，才能控制查询的正确性和性能。

**5．Jakarta Persistence. API и интеграция с ORM-провайдерами — Jakarta Persistence、API 与 ORM Provider 集成**

Jakarta Persistence, или JPA, — это спецификация для работы с постоянными данными и объектно-реляционным отображением. Она определяет сущности, их состояния, связи, контекст

персистентности и стандартные операции с данными.

Jakarta Persistence，也称 JPA，是用于持久化数据访问和对象关系映射的规范。它定义实体、实体状态、关联关系、持久化上下文以及标准数据操作。

Основные интерфейсы — EntityManagerFactory и EntityManager. EntityManagerFactory создаёт EntityManager, который управляет сущностями в контексте персистентности. Метод persist сохраняет новую сущность, find находит её по первичному ключу, а remove помечает управляемую сущность для удаления.

主要接口是 EntityManagerFactory 和 EntityManager。EntityManagerFactory 创建 EntityManager，后者管理持久化上下文中的实体。persist 方法用于持久化新实体，find 根据主键查找实体，remove 将受管理实体标记为删除。

Для запросов используются JPQL, Criteria API и при необходимости SQL. JPQL работает с сущностями и их атрибутами, а SQL — с таблицами и столбцами базы данных. Сама спецификация JPA не выполняет SQL. Для этого нужен провайдер, например EclipseLink или Hibernate.

查询可以使用 JPQL、Criteria API，以及必要时使用 SQL。JPQL 面向实体及其属性，SQL 面向数据库表及列。JPA 规范本身不执行 SQL，需要 EclipseLink 或 Hibernate 等 Provider 来完成。

В моей работе настройки единиц персистентности находятся в persistence.xml. Компонент Database управляет EntityManager и транзакциями. Используется RESOURCE_LOCAL, поэтому транзакция явно начинается и завершается в методе write.

我的实验在 persistence.xml 中配置持久化单元。Database 组件管理 EntityManager 和事务。项目使用 RESOURCE_LOCAL，因此事务在 write 方法中显式开始和结束。

Flush синхронизирует изменения с базой данных, но не завершает транзакцию. Commit фиксирует транзакцию. До фиксации изменения могут быть отменены откатом.

flush 将修改同步到数据库，但不结束事务。commit 提交事务。在提交前，修改可以通过回滚撤销。

**6．Технология Jakarta Data — Jakarta Data 技术**

Jakarta Data — это спецификация, упрощающая доступ к данным с помощью репозиториев. Разработчик объявляет интерфейс репозитория и методы доступа к данным, а реализацию предоставляет Jakarta Data-провайдер.

Jakarta Data 是通过 Repository 简化数据访问的规范。开发者声明 Repository 接口和数据访问方法，由 Jakarta Data Provider 提供实现。

Поддерживаются стандартные операции с данными, запросы и пагинация. В зависимости от провайдера можно работать с разными типами хранилищ.

它支持标准数据操作、查询和分页。根据 Provider 的不同，可以访问不同类型的数据存储。

Jakarta Persistence описывает работу с сущностями и контекстом персистентности, а Jakarta Data предоставляет более высокоуровневую абстракцию репозитория. Jakarta Data не заменяет Jakarta Persistence и может использовать её через подходящий провайдер.

Jakarta Persistence 描述实体和持久化上下文的操作，而 Jakarta Data 提供更高层次的 Repository 抽象。Jakarta Data 不会取代 Jakarta Persistence，并且可以通过合适的 Provider 使用它。

В моей работе Jakarta Data не используется. Запросы реализованы вручную через EntityManager и JPQL.

我的实验没有使用 Jakarta Data，而是通过 EntityManager 和 JPQL 手动实现查询。

**7．Spring. Сходства и отличия с Java EE — Spring 与 Java EE 的异同**

Spring Framework — это модульный фреймворк для разработки Java-приложений. Его основой является IoC-контейнер, который создаёт компоненты и внедряет зависимости. Spring также предоставляет средства

для веб-приложений, транзакций, доступа к данным и аспектно-ориентированного программирования.

Spring Framework 是用于开发 Java 应用的模块化框架。它的基础是负责创建组件和注入依赖的 IoC 容器。Spring 还提供 Web 应用、事务、数据访问和面向切面编程方面的能力。

Инверсия управления означает, что создание и связывание компонентов передаётся контейнеру. Компонент получает необходимые зависимости извне вместо того, чтобы самостоятельно создавать их.

控制反转意味着组件的创建和组装交给容器。组件从外部获得所需依赖，而不是自己创建依赖。

Spring и Jakarta EE решают похожие задачи: управление компонентами, внедрение зависимостей и организация серверной логики. Основное различие заключается в том, что Jakarta EE — это набор стандартизованных спецификаций с разными реализациями, а Spring предоставляет собственную модель программирования и конкретные библиотеки.

Spring 与 Jakarta EE 解决相近的问题，包括组件管理、依赖注入和后端逻辑组织。主要区别在于，Jakarta EE 是具有不同实现的一组标准化规范，而 Spring 提供自身的编程模型和具体库。

Эти технологии могут взаимодействовать. Например, Spring поддерживает Jakarta Persistence и Servlet API. При этом Spring не является реализацией всей платформы Jakarta EE.

两者可以协作。例如，Spring 支持 Jakarta Persistence 和 Servlet API。但 Spring 不是整个 Jakarta EE 平台的实现。

В моей работе используются Jakarta EE и CDI. Spring в проекте не используется.

我的实验使用 Jakarta EE 和 CDI，没有使用 Spring。

**8．Spring Boot — Spring Boot**

Spring Boot — это проект на основе Spring Framework, который упрощает создание и запуск Spring-приложений. Он предоставляет автоматическую конфигурацию, стартеры зависимостей и согласованное управление их версиями.

Spring Boot 是基于 Spring Framework、用于简化 Spring 应用创建和运行的项目。它提供自动配置、依赖 Starter 和协调一致的依赖版本管理。

Стартер позволяет удобно подключить набор зависимостей для определённой задачи, например для веб-приложения. Автоматическая конфигурация учитывает библиотеки в classpath, настройки приложения и уже объявленные бины.

Starter 便于引入完成某项任务所需的一组依赖，例如开发 Web 应用的依赖。自动配置会考虑 classpath 中的库、应用配置以及已经声明的 Bean。

Веб-приложение часто можно запустить как исполняемый JAR со встроенным сервером. Для проверки состояния и получения метрик приложения можно использовать Actuator.

Web 应用通常可以作为带有内嵌服务器的可执行 JAR 启动。可以使用 Actuator 检查应用状态并获取指标。

Spring Framework предоставляет основные механизмы, а Spring Boot упрощает их настройку, объединение и запуск приложения. Spring Boot сокращает объём инфраструктурной конфигурации, но не заменяет бизнес-логику.

Spring Framework 提供基础机制，Spring Boot 简化这些机制的配置、组合和应用启动。Spring Boot 减少基础设施配置工作，但不会替代业务逻辑。

Моя работа запускается как WAR-приложение в Payara. Spring Boot в ней не используется.

我的实验作为 WAR 应用在 Payara 中运行，没有使用 Spring Boot。

**9．Spring Data — Spring Data**

Spring Data — это семейство проектов, упрощающих доступ к данным в приложениях Spring. Оно предоставляет абстракцию репозитория, поддержку запросов, сортировки и пагинации. Существуют модули для разных технологий, например Spring Data JPA, JDBC и MongoDB.

Spring Data 是用于简化 Spring 应用数据访问的一组项目。它提供 Repository 抽象，以及查询、排序和分页支持。它为不同技术提供模块，例如 Spring Data JPA、JDBC 和 MongoDB。

В Spring Data JPA разработчик обычно объявляет интерфейс, наследующий JpaRepository. Реализацию предоставляет инфраструктура Spring Data. Запросы можно определять по именам методов или с помощью аннотации Query.

在 Spring Data JPA 中，开发者通常声明继承 JpaRepository 的接口，由 Spring Data 基础设施提供实现。查询可以根据方法名定义，也可以通过 Query 注解定义。

Spring Data JPA не является ORM-провайдером. Ему требуется реализация JPA, например Hibernate или EclipseLink.

Spring Data JPA 不是 ORM Provider，它需要 Hibernate 或 EclipseLink 等 JPA 实现。

Spring Data и Jakarta Data используют репозитории для упрощения доступа к данным. Однако Jakarta Data — это стандартная спецификация, а Spring Data — семейство проектов экосистемы Spring. Их API и механизмы реализации различаются.

Spring Data 与 Jakarta Data 都使用 Repository 简化数据访问。但是，Jakarta Data 是标准规范，Spring Data 是 Spring 生态中的项目族。它们的 API 和实现机制不同。

В моей работе Spring Data не используется. Операции реализованы через EntityManager в сервисном слое.

我的实验没有使用 Spring Data，而是在 Service 层通过 EntityManager 实现数据操作。

**概括自己的实验 / Краткое описание моей реализации**

В моей лабораторной работе используется Jakarta EE. Приложение запускается в Payara. Компоненты управляются CDI-контейнером, а HTTP-запросы обрабатываются REST-ресурсами. Бизнес-логика находится в сервисном слое. Для работы с PostgreSQL используется стандартный API Jakarta Persistence и провайдер EclipseLink. Spring, Spring Boot, Spring Data и Jakarta Data в этой реализации не используются.

我的实验使用 Jakarta EE。应用运行在 Payara 中。组件由 CDI 容器管理，HTTP 请求由 REST 资源处理。业务逻辑位于 Service 层。系统通过标准 Jakarta Persistence API 和 EclipseLink Provider 访问 PostgreSQL。当前实现没有使用 Spring、Spring Boot、Spring Data 和 Jakarta Data。